```
# Maze Game
# The maze is represented by an 8x8 grid A = [a00 ... a77]
# 1 = free cell, 0 = wall
from RPLCD.i2c import CharLCD
import smbus
import time
import RPi.GPIO as GPIO
import paho.mqtt.client as mqtt
import json

ADC_ADDR = 0x4b
bus = smbus.SMBus(1)

lcd = CharLCD(
   i2c_expander='PCF8574',
   address=0x27,
   port=1,
   cols=16,
   rows=2,
   auto_linebreaks=True
)

RED = 17 #green
GREEN = 27 #red
BLUE = 22
BUTTON_PIN = 13
BUZZER_PIN = 18

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(RED, GPIO.OUT)
GPIO.setup(GREEN, GPIO.OUT)
GPIO.setup(BLUE, GPIO.OUT)
GPIO.setup(BUTTON_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(BUZZER_PIN, GPIO.OUT)
mqtt_Client = mqtt.Client()
mqtt_Client.connect("localhost",1883,60)

#Create PWM objects
red_pwm = GPIO.PWM(RED, 1000)
green_pwm = GPIO.PWM(GREEN, 1000)
blue_pwm = GPIO.PWM(BLUE, 1000)

#Start PWM
red_pwm.start(0)
green_pwm.start(0)
blue_pwm.start(0)
```

```
def color_flash():
    for r, g, b in [(100,0,0), (0,100,0), (0,0,100)]:
        red_pwm.ChangeDutyCycle(r)
        green_pwm.ChangeDutyCycle(g)
        blue_pwm.ChangeDutyCycle(b)
        time.sleep(0.5)

def lcd_show(msg: str):
    lcd.clear()
    lcd.write_string(msg)

def light_color(color, duration=0.5):
    if color == "red":
        red_pwm.ChangeDutyCycle(100)
        green_pwm.ChangeDutyCycle(0)
        blue_pwm.ChangeDutyCycle(0)
    elif color == "green":
        red_pwm.ChangeDutyCycle(0)
        green_pwm.ChangeDutyCycle(100)
        blue_pwm.ChangeDutyCycle(0)
    elif color == "blue":
        red_pwm.ChangeDutyCycle(0)
        green_pwm.ChangeDutyCycle(0)
        blue_pwm.ChangeDutyCycle(100)
    time.sleep(duration)
    red_pwm.ChangeDutyCycle(0)
    green_pwm.ChangeDutyCycle(0)
    blue_pwm.ChangeDutyCycle(0)
def pip(duration=0.1):
    GPIO.output(BUZZER_PIN, GPIO.HIGH)
    time.sleep(duration)
    GPIO.output(BUZZER_PIN, GPIO.LOW)

X_CH = 0   # Joystick X axis (A0)
Y_CH = 4   # Joystick Y axis (A4)

def read_adc(ch):
    cmd = 0x84 | (ch << 4)
    bus.write_byte(ADC_ADDR, cmd)
    time.sleep(0.001)
    return bus.read_byte(ADC_ADDR)

def get_move():
    while True:
        # Check button first (hint request)
        if GPIO.input(BUTTON_PIN) == GPIO.LOW:
            # wait for button release
            while GPIO.input(BUTTON_PIN) == GPIO.LOW:
```

```
                time.sleep(0.01)
            return "hint"

        x = read_adc(X_CH)
        y = read_adc(Y_CH)

        # Detect movement
        if x < 80:
            move = "left"
        elif x > 170:
            move = "right"
        elif y < 80:
            move = "up"
        elif y > 170:
            move = "down"
        else:
            continue  # joystick still centered
        # Wait until joystick returns to center
        while True:
            x = read_adc(X_CH)
            y = read_adc(Y_CH)
            if 80 <= x <= 170 and 80 <= y <= 170:
                break
            time.sleep(0.01)

        time.sleep(0.1)  # debounce
        return move


def hitwall():
    lcd_show("Wall!!!")
    light_color("red")
    pip(duration=0.5)
    time.sleep(0.1)



A = [
    [1, 1, 0, 1, 0, 0, 1, 0],
    [1, 0, 0, 1, 1, 1, 1, 0],
    [1, 1, 0, 1, 1, 0, 1, 0],
    [1, 1, 1, 1, 0, 0, 1, 0],
    [0, 0, 1, 1, 1, 1, 1, 1],
    [1, 0, 0, 1, 0, 0, 0, 1],
    [1, 1, 0, 1, 0, 1, 0, 1],
    [0, 1, 1, 1, 0, 1, 1, 1],
]
```

```
start_state = "a73"
finish_state = "a03"
current_state = start_state

steps = 0
hint_counter = 0
MAX_HINTS = 3

def get_indices(state):
   i = int(state[1])
   j = int(state[2])
   return i, j

def possible_moves(state):
   i, j = get_indices(state)
   moves = []
   if i > 0 and A[i - 1][j] == 1:
       moves.append("up")
   if i < 7 and A[i + 1][j] == 1:
       moves.append("down")
   if j > 0 and A[i][j - 1] == 1:
       moves.append("left")
   if j < 7 and A[i][j + 1] == 1:
       moves.append("right")
   return moves

def next_state(state, move):
   i, j = get_indices(state)
   if move == "up":
       i -= 1
   elif move == "down":
       i += 1
   elif move == "left":
       j -= 1
   elif move == "right":
       j += 1
   return f"a{i}{j}"

def publish_state():
   mqtt_Client.publish("maze/state", current_state)


############################
# GAME LOOP
############################
publish_state()
mqtt_Client.publish("joystick/move",  " ")
lcd_show("Game on!")
```

```
light_color("blue",0.5 )

while True:
   print(f"\nSteps: {steps}")
   print(f"current state: {current_state}")
   print("Enter move (up, down, left, right):")

   move = get_move()
   if move in ["up", "down", "left", "right"]:
       mqtt_Client.publish("joystick/move",  move)
   if move == "hint":
       if hint_counter < MAX_HINTS:
           lcd_show(f"Hints:{possible_moves(current_state)}")
           light_color("blue",0.2)
           print(f"Hints:{possible_moves(current_state)}")
          hint_counter += 1
       else:
           lcd_show("No more hints")
           light_color("red",0.2)
           print("No more hints")
       time.sleep(0.3)
       continue


   if move not in possible_moves(current_state):
       hitwall()
       steps += 1
       print("Wall! State unchanged.")
   current_state = next_state(current_state, move)
   steps += 1
   publish_state()
   lcd_show(f"Steps: {steps}")
   light_color("green",0.2)
   pip(0.2)
   time.sleep(0.1)

   if current_state == finish_state:
       lcd_show(f"Finish! Steps:{steps}")
       print("\nFinish!")
       print(f"Total steps taken: {steps}")
       color_flash()
       break
```